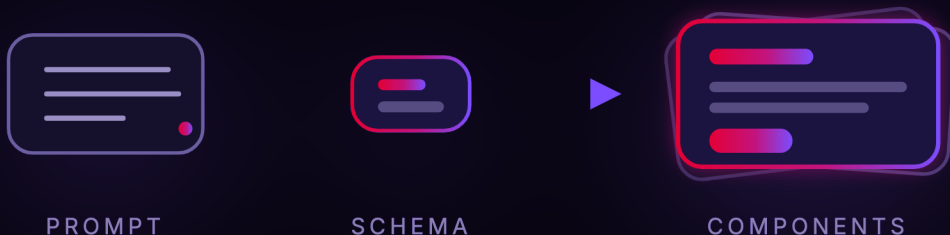


More Than Meets the Eye

Building UIs that Transform



Powered by Angular Dynamic Components

Maina Wycliffe · Angular + Genkit + Gemini 3 Flash

The problem

Chatbots answer in **text**.

Real tasks need real **interactive UI**:

-  *pick* a flight from a comparison
-  *choose* a seat on a map
-  *fill in* passenger details
-  *pay*

A wall of markdown can't do any of that. **Text-only AI UX hits a ceiling.**

A2UI = Agent-to-UI

Let the agent decide **which component** to render — not just the words.

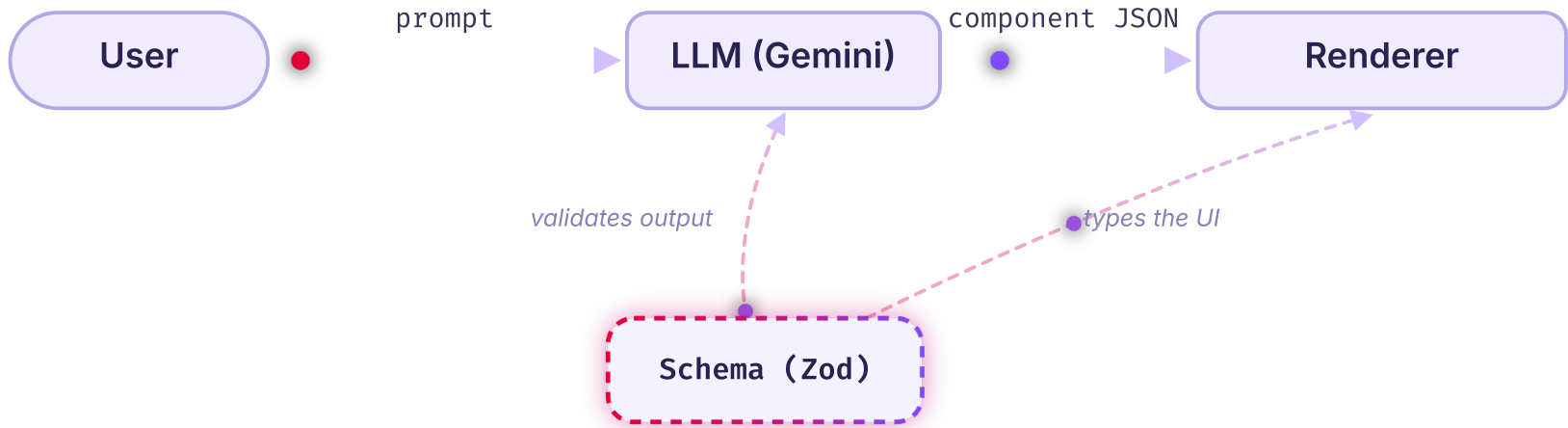
Server-Driven UI, taken to the LLM extreme:

- The agent returns a typed **render instruction**
- The frontend looks it up and **materializes** the component
- The component sends user interactions **back** to the agent
- Repeat until the task is done

"A2UI" here is a pattern in this repo, not an external spec — but the idea generalizes.

```
{  
  "kind": "component",  
  "instruction": {  
    "component": "FlightComparisonCard",  
    "message": "Here are 3 options:",  
    "props": { "origin": "Nairobi", ... }  
  }  
}
```

Architecture



One Zod schema is the contract for *both* ends — imported into the frontend via the `@shared/*` path alias.

The contract: one Zod schema, both ends

`server/schema.ts` is the single source of truth. `.describe()` hints become the LLM's blueprint.

```
export const RenderInstruction = z.discriminatedUnion('component', [
  z.object({
    component: z.literal('FlightComparisonCard'),
    message: z.string(),
    props: FlightComparisonProps,
  }),
  // ...SeatMapCard, PassengerFormCard, PaymentCard, BookingConfirmationCard, DataTableCard
]);

export const AgentResponse = z.discriminatedUnion('kind', [
  z.object({ kind: z.literal('text'), text: z.string() }),
  z.object({ kind: z.literal('component'), instruction: RenderInstruction }),
]);

// Types flow back to the frontend, for free:
export type ComponentName = RenderInstruction['component'];
```

Genkit calls Gemini with `output: { schema: AgentResponse }` — structured output, validated before it ever reaches Angular.

Structured output — any framework

Same idea everywhere: hand the model a **schema**, get back schema-valid JSON.

Genkit · this repo

```
await ai.generate({
  output: {
    schema: AgentResponse,
  },
});
```

Zod schema · TS

Google ADK · Python

```
LlmAgent(
  model="gemini-2.5-flash",
  output_schema=AgentResponse,
)
```

Pydantic model

LangChain · TS

```
model.withStructuredOutput(
  AgentResponse,
);
```

Zod schema · TS

Different SDKs, one contract — a typed schema becomes the model's output guarantee.

Modern Angular toolkit

Every card is built with today's Angular. Straight from `flight-comparison-card.ts`:

```
import { Component, inject, input, signal } from '@angular/core';
import { DecimalPipe } from '@angular/common';

@Component({
  selector: 'app-flight-comparison-card',
  template: `
    @if (highlightUnder()) { ... }
    @for (f of flights(); track f.id) { ... }    <!-- built-in control flow -->
  `,
  imports: [DecimalPipe],                      // standalone - no NgModule
})
export class FlightComparisonCard {
  origin = input.required<Props['origin']>();    // signal inputs
  flights = input.required<Props['flights']>();
  highlightUnder = input<Props['highlightUnder']>(undefined);

  selecting = signal(false);                    // writable signal
  private bus = inject(AgentBus);               // inject(), not constructor
}
```

The Angular you know and love

Dynamic components — the core

Two pieces bridge a **string** from the agent to a **rendered component**.

1. The allow-listed registry — `component-registry.ts`

```
export const COMPONENT_REGISTRY:
  Record<ComponentName, Type<unknown>> = {
  FlightComparisonCard,
  SeatMapCard,
  PassengerFormCard,
  PaymentCard,
  BookingConfirmationCard,
  DataTableCard,
};
```

Typed by `ComponentName` → the compiler **forces** every schema variant to have an entry.

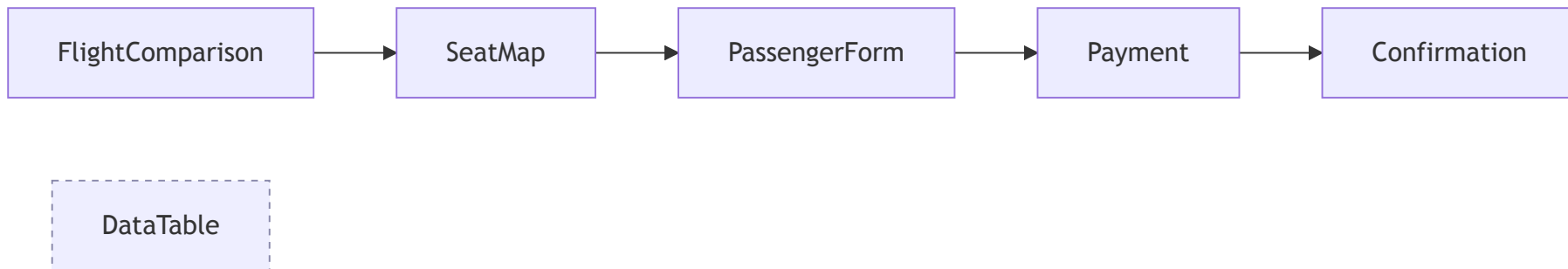
2. `NgComponentOutlet` — `ai-ui-host.ts`

```
<ng-container
  *ngComponentOutlet="
    componentType(inst.component);
    inputs: inst.props
  " />
```

`inputs:` binds the agent's `props` straight to each card's signal inputs — typed via `Extract<RenderInstruction, {component:'...'}>['props']`.

The agent can only ever render a component on the allow-list. 🔒

The cards = the booking flow



Five cards walk the flow, one per step. Each name maps **1:1** to a schema variant and a registry entry.

...plus **DataTableCard** — rendered *on-demand*, not part of the sequence.

Spotlight: building a new card

Adding `DataTableCard` (a generic, sortable + expandable table) took 3 steps:

1 · Schema variant `server/schema.ts`

```
z.object({
  component:
    z.literal('DataTableCard'),
  message: z.string(),
  props: DataTableProps,
})
```

2 · Registry entry `component-registry.ts`

```
COMPONENT_REGISTRY = {
  ...,
  DataTableCard,
}
```

(compiler demands it)

3 · The component `data-table-card.ts`

```
columns = input.required<...>();
rows     = input.required<...>();
```

Sorting & expansion are **pure local state** — no round-trip. `computed()` derives the view:

```
sortIndex = signal<number | null>(null); // sort + expand: local signals
sortedRows = computed(() => /* sort this.rows() by sortIndex */);
```



DEMO — "compare those flights in a table", then sort a column & expand a row

Closing the loop: AgentBus

Dynamically-created components can't wire `@Output()` to the host — so a tiny RxJS bus carries typed events.

```
// agent-bus.ts
export type AgentEvent =
  | { type: 'FLIGHT_SELECTED'; ... }
  | { type: 'SEAT_SELECTED'; ... }
  | { type: 'PASSENGER_SUBMITTED'; ... }
  | { type: 'PAYMENT_CONFIRMED'; ... };

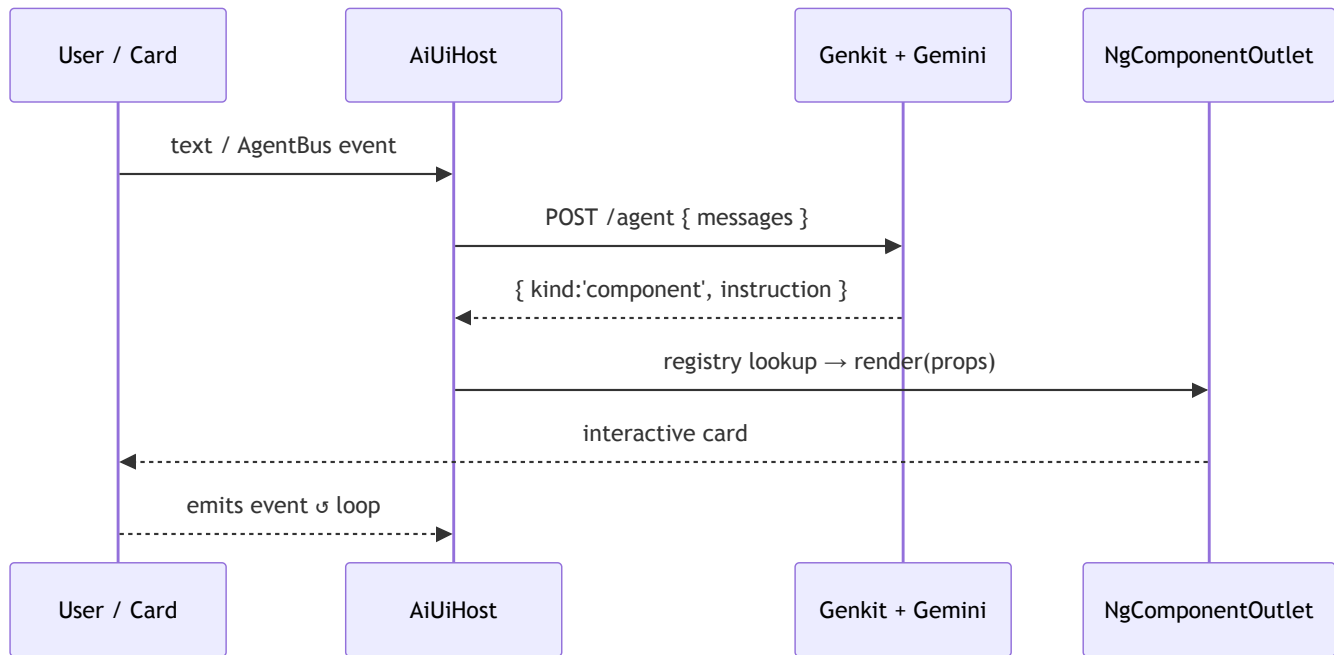
@Injectable({ providedIn: 'root' })
export class AgentBus {
  readonly events = new Subject<AgentEvent>();
  emit(e: AgentEvent) { this.events.next(e); }
}
```

The host forwards each event as the **next user turn**:

```
// ai-ui-host.ts
this.bus.events.subscribe(e => this.sendEvent(e));
```

Counter-example: `DataTableCard` is presentational — sort/expand never touch the bus. Not every interaction needs the agent.

One full turn



DEMO — "Fly Nairobi to Mombasa Friday under 8,000 KES" → open the JSON drawer

Takeaways

- 🧠 The LLM describes the UI
- 🛡️ We enforce & validate via schema
- 🅐 We build native Angular components
- 🎨 Consistent design language & Angular best practice



Thank you 🙏

github.com/mainawycliffe/ng-a2ui